

ROBOTICS SUMMER SCHOOL - FUNDAMENTALS

Mathematics for Robotics

Linear algebra

Krish Pandya

Morning session

Vectors

A vector $\mathbf{v} \in \mathbb{R}^n$ is an ordered list of n real numbers. It describes a point or a direction in n -dimensional space.

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

In robotics every quantity with multiple components is a vector: position (x, y, z) , velocity, angular rate, LiDAR point, pixel coordinate. The number of components is the dimension; it equals the degrees of freedom of the quantity being described.

The hat notation $\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$ denotes a unit vector. It carries direction but discards magnitude.

```
import numpy as np

v = np.array([1.0, 2.0, 3.0])

# L2 norm (Euclidean length)
print(np.linalg.norm(v))      # 3.742

# Unit vector
v_hat = v / np.linalg.norm(v)
print(np.linalg.norm(v_hat))  # 1.0
```

A column vector in $\mathbb{R}^n$ is a special case of a matrix in $\mathbb{R}^{n \times 1}$. Every result for matrices applies to vectors.

Vector operations

ADDITION AND SCALING

Addition is component-wise. Scaling multiplies every component by a scalar.

$$(\mathbf{u} + \mathbf{v})_i = u_i + v_i, \quad (\alpha \mathbf{v})_i = \alpha v_i$$

DOT PRODUCT

The dot product maps two vectors to a scalar.

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

When $\mathbf{u} \cdot \mathbf{v} > 0$, the vectors point into the same half-space.

When $\mathbf{u} \cdot \mathbf{v} = 0$, the vectors are orthogonal. When $\mathbf{u} \cdot \mathbf{v} < 0$, they oppose each other.

CROSS PRODUCT (IN $\mathbb{R}^3$ ONLY)

The cross product produces a vector orthogonal to both inputs.

$$\mathbf{u} \times \mathbf{v} = \begin{pmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{pmatrix}$$

Its magnitude is the area of the parallelogram spanned by $\mathbf{u}$ and $\mathbf{v}$.

$$\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$$

Angular momentum is $\mathbf{L} = \mathbf{r} \times \mathbf{p}$ and torque is $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$. Both use the cross product to encode rotational quantities from linear ones.

Norms and distance

A norm $\|v\|$ measures the size of a vector. Different norms measure size differently and produce different behaviour in optimization.

$$\|v\|_1 = \sum_i |v_i| \quad \text{sum of absolute values}$$

$$\|v\|_2 = \sqrt{\sum_i v_i^2} \quad \text{Euclidean length}$$

$$\|v\|_\infty = \max_i |v_i| \quad \text{largest component}$$

The L2 norm corresponds to physical distance. It is the default in robotics.

```
v = np.array([3.0, 4.0])  
  
print(np.linalg.norm(v, ord=1))      # 7.0  
print(np.linalg.norm(v, ord=2))      # 5.0  
print(np.linalg.norm(v, ord=np.inf))  # 4.0
```

Minimizing the L1 norm produces sparse solutions: many components are driven to exactly zero. This is useful in compressed sensing and feature selection. Minimizing the L2 norm produces smooth solutions with all components non-zero.

For matrices, the Frobenius norm extends L2:

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}$$

Orthogonality and projection

Two vectors are orthogonal when $\mathbf{u} \cdot \mathbf{v} = 0$.

PROJECTION

The projection of $\mathbf{v}$ onto $\mathbf{u}$ is the component of $\mathbf{v}$ in the direction of $\mathbf{u}$.

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}} \mathbf{u}$$

This decomposes $\mathbf{v}$ into two orthogonal parts: one along $\mathbf{u}$, one perpendicular to $\mathbf{u}$.

ORTHONORMAL BASIS

A set of vectors is orthonormal if every pair is orthogonal and every vector has unit length: $\mathbf{e}_i \cdot \mathbf{e}_j = \delta_{ij}$.

In an orthonormal basis, expressing any vector is just a sequence of dot products.

$$\mathbf{v} = \sum_i (\mathbf{v} \cdot \mathbf{e}_i) \mathbf{e}_i$$

No matrix inversion needed. This is why orthonormal bases appear everywhere in numerical computation.

Kalman filter updates work by projecting the prediction error onto the sensor measurement space. The Kalman gain matrix weights this projection by the sensor noise covariance from probability lecture.

Two vectors satisfy $u \cdot v = 0$. What is the angle between them? If $u = (1, 0, 0)$, name two vectors orthogonal to it.

Matrices: notation and shape

A matrix $A \in \mathbb{R}^{m \times n}$ is a rectangular array with m rows and n columns. Multiplying Ax maps a vector in $\mathbb{R}^n$ to a vector in $\mathbb{R}^m$. Every matrix is a linear function between vector spaces.

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}$$

The transpose A^T swaps rows and columns: $(A^T)_{ij} = A_{ji}$. For real matrices, $(A^T)^T = A$ and $(AB)^T = B^T A^T$.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3): R^3 -> R^2

x = np.array([1.0, 0.0, 0.0])
print(A @ x)           # [1. 4.] -- first column of A

print(A.T.shape)       # (3, 2)
print((A @ A.T).shape) # (2, 2) -- square, symmetric
print((A.T @ A).shape) # (3, 3) -- square, symmetric
```

A camera matrix maps 3D world points in $\mathbb{R}^4$ (homogeneous) to 2D image points in $\mathbb{R}^3$ (homogeneous). It is a 3×4 matrix.

Matrix multiplication

For $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, the product $C = AB \in \mathbb{R}^{m \times n}$ has entries:

$$C_{ij} = \sum_{l=1}^k A_{il} B_{lj}$$

The inner dimensions must match. The result shape comes from the outer dimensions.

Properties:

- Not commutative: $AB \neq BA$ in general.
- Associative: $(AB)C = A(BC)$.
- Distributive: $A(B + C) = AB + AC$.
- Transpose of product: $(AB)^T = B^T A^T$.

MATRIX PRODUCT AS COMPOSED TRANSFORMATIONS

AB applies B first, then A . Order matters because transformations do not generally commute.

```
# Rotate 90 degrees
R = np.array([[0, -1], [1, 0]])
# Scale by 2
S = np.array([[2, 0], [0, 2]])

v = np.array([1.0, 0.0])

# Scale then rotate
print(R @ (S @ v)) # [-0. 2.]
# Rotate then scale -- same here because S = 2I
print(S @ (R @ v)) # [-0. 2.]

# Non-commutative example
T = np.array([[1, 1], [0, 1]]) # shear
print(R @ T) # != T @ R
```


Special matrices

IDENTITY: I

Ones on the diagonal, zeros elsewhere. $AI = IA = A$ for any compatible A .

DIAGONAL: D

Non-zero entries only on the diagonal. Multiplication scales each coordinate independently: $(Dx)_i = D_{ii}x_i$.

SYMMETRIC: $A = A^T$

Covariance matrices are always symmetric. Symmetric matrices have real eigenvalues and mutually orthogonal eigenvectors.

ORTHOGONAL: $Q^T Q = I$

The columns are orthonormal. Multiplication by Q preserves lengths and angles. The inverse is free: $Q^{-1} = Q^T$.

Rotation matrices are orthogonal with $\det(Q) = +1$.

POSITIVE DEFINITE: $x^T A x > 0$ FOR ALL $x \neq 0$

All eigenvalues are positive. Covariance matrices are positive semi-definite. Their inverses are the information matrices from probability lecture.

Orthogonal matrices preserve geometry. Symmetric positive definite matrices appear wherever there is uncertainty, energy, or a metric.

Rank and null space

The column space of A is the set of all vectors A can produce. Its dimension is the rank.

$$\text{rank}(A) = \dim(\text{col}(A))$$

The null space is the set of inputs A maps to zero.

$$\text{null}(A) = \{x : Ax = 0\}$$

The rank-nullity theorem connects them:

$$\text{rank}(A) + \dim(\text{null}(A)) = n$$

A matrix is full rank when $\text{rank}(A) = \min(m, n)$. A rank-deficient matrix collapses at least one dimension of the input.

```
A = np.array([[1, 2, 3],
              [2, 4, 6],  # row 2 = 2 * row 1
              [0, 1, 1]])

print(np.linalg.matrix_rank(A))  # 2, not 3

# Null space: right singular vectors for zero singular values
_, s, Vt = np.linalg.svd(A)
null_mask = s < 1e-10
null_space = Vt[null_mask].T
print(null_space.shape)  # (3, 1) -- one null vector
```

In camera calibration the null space of the measurement matrix gives the camera parameters. In SLAM the null space encodes the unobservable absolute position and scale (the gauge freedom).

Linear systems

The system $Ax = b$ asks: which input x does A map to b ?

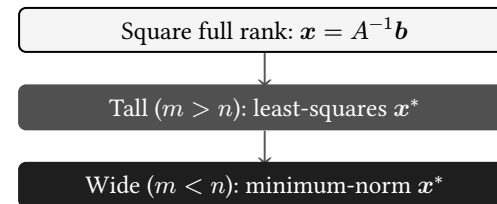
Three cases determine the answer.

- b is not in the column space of A : no solution.
- b is in the column space and $\dim(\text{null}(A)) = 0$: one solution.
- b is in the column space and $\dim(\text{null}(A)) > 0$: infinitely many solutions.

When A is square and full rank: $x = A^{-1}b$.

When A is tall ($m > n$, over-determined): no exact solution in general; minimize $\|Ax - b\|_2^2$ (least squares).

When A is wide ($m < n$, under-determined): pick the minimum-norm solution.



Most estimation problems are over-determined: many noisy measurements constrain few unknowns. Least squares is the standard answer.

$Ax = 0$ always has $x = 0$ as a solution. When does it have a non-trivial solution? What does that say about the rank of A ?

Determinant

The determinant of a square matrix measures how the matrix scales volume.

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

$|\det(A)|$ is the volume of the image of the unit cube under A . When $\det(A) = 0$, the matrix collapses space to a lower dimension and is not invertible.

Key properties:

- $\det(A) = 0$ if and only if A is rank-deficient.
- $\det(AB) = \det(A) \det(B)$.
- $\det(A^T) = \det(A)$.
- $\det(A^{-1}) = \frac{1}{\det(A)}$.

The sign encodes orientation. A positive determinant means the transformation preserves the handedness of space. A negative determinant reflects it.

Rotation matrices have $\det(R) = +1$. They preserve volume and orientation, which is why they represent physically realizable rotations.

Reflection matrices have $\det = -1$.

Computing a large determinant is numerically expensive and can overflow or underflow. Checking rank via singular value decomposition is the preferred approach in practice.

Matrix inverse and condition number

For a square full-rank matrix A , the inverse A^{-1} satisfies:

$$AA^{-1} = A^{-1}A = I$$

Useful identities:

$$(AB)^{-1} = B^{-1}A^{-1}$$

$$(A^T)^{-1} = (A^{-1})^T$$

For orthogonal Q : $Q^{-1} = Q^T$, so inversion is just a transpose. Rotation matrices are always cheap to invert.

Direct matrix inversion is numerically avoided in practice. Solve $A\mathbf{x} = \mathbf{b}$ via $\mathbf{x} = \text{np.linalg.solve}(A, \mathbf{b})$ instead of $\text{inv}(A) @ \mathbf{b}$.

CONDITION NUMBER

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

A large condition number means small errors in $\mathbf{b}$ produce large errors in $\mathbf{x} = A^{-1}\mathbf{b}$. Each decade of κ costs roughly one digit of accuracy in the solution.

```
A_good = np.eye(2)
A_bad  = np.array([[1, 1], [1, 1.0001]])

print(np.linalg.cond(A_good)) # 1.0
print(np.linalg.cond(A_bad))  # ~40000
```

Bundle adjustment fails when the measurement matrix is ill-conditioned. Normalizing input data (zero-mean, unit scale) reduces the condition number.

A matrix has condition number $\kappa = 10^{12}$. A 64-bit float has about 16 significant digits. How many digits of the solution $x = A^{-1}b$ can you trust?

Eigenvalues and eigenvectors

A non-zero vector $\mathbf{v}$ is an eigenvector of A when:

$$A\mathbf{v} = \lambda\mathbf{v}$$

The scalar λ is the eigenvalue. The matrix changes the length of $\mathbf{v}$ by λ but does not change its direction (up to sign).

Eigenvalues satisfy the characteristic equation:

$$\det(A - \lambda I) = 0$$

An $n \times n$ matrix has at most n eigenvalues. Real matrices can have complex eigenvalues that come in conjugate pairs.

```
A = np.array([[3.0, 1.0],
              [0.0, 2.0]])

vals, vecs = np.linalg.eig(A)
print(vals)      # [3. 2.]
print(vecs)      # columns are eigenvectors

# Verify A v = lambda v
v = vecs[:, 0]
lam = vals[0]
print(np.allclose(A @ v, lam * v)) # True
```

PCA finds eigenvectors of the data covariance matrix. The eigenvectors are the directions of maximum variance; the eigenvalues are the variances. This is eigendecomposition applied to data.

Properties of eigenvalues

SYMMETRIC MATRICES: $A = A^T$

All eigenvalues are real. Eigenvectors for distinct eigenvalues are orthogonal. The matrix always has an orthogonal eigendecomposition:

$$A = Q\Lambda Q^T, \quad Q^T Q = I$$

POSITIVE DEFINITE MATRICES

All eigenvalues satisfy $\lambda_i > 0$. The quadratic form $\mathbf{x}^T A \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$. This is the matrix generalization of a positive number.

TRACE AND DETERMINANT

$$\text{tr}(A) = \sum_i A_{ii} = \sum_i \lambda_i$$

$$\det(A) = \prod_i \lambda_i$$

ORTHOGONAL MATRICES: $Q^T Q = I$

All eigenvalues have absolute value 1: $|\lambda_i| = 1$. In $\mathbb{R}$ they are ± 1 . In $\mathbb{C}$ they lie on the unit circle. A 3D rotation matrix has one real eigenvalue of +1 (the rotation axis) and two complex conjugate eigenvalues.

Reading the eigenvalue structure tells you what a matrix does to space: positive eigenvalues stretch, negative ones flip, zero ones collapse, and complex ones rotate. 🐱

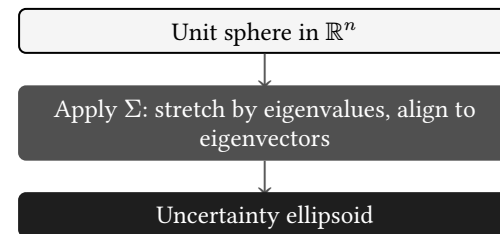
Geometric view: the uncertainty ellipsoid

Every symmetric positive definite matrix A defines an ellipsoid through its eigenvectors and eigenvalues.

The eigenvectors give the axes of the ellipsoid. The eigenvalues give the squared half-lengths along those axes.

Applying A to the unit sphere stretches it into this ellipsoid. A large eigenvalue means A amplifies strongly in that direction. A small eigenvalue means A compresses in that direction.

The covariance matrix Σ from probability lecture is always a positive semi-definite symmetric matrix. Its eigenvectors are the principal axes of the uncertainty ellipsoid; its eigenvalues are the variances along those axes.



A diagonal Σ with equal entries is a sphere: isotropic uncertainty. A large off-diagonal entry tilts the ellipsoid: position error in x implies correlated error in y . This is the covariance geometry from probability lecture.

Eigendecomposition

A diagonalizable matrix A factors as:

$$A = V\Lambda V^{-1}$$

where the columns of V are eigenvectors and Λ is diagonal with eigenvalues. For symmetric A , V is orthogonal ($V^{-1} = V^T$):

$$A = Q\Lambda Q^T$$

Matrix powers and the matrix exponential become direct:

$$A^k = Q\Lambda^k Q^T, \quad e^A = Qe^\Lambda Q^T$$

The matrix exponential appears in rigid body motion: integrating an angular velocity matrix gives a rotation matrix.

```
# Use eig for symmetric matrices (real eigenvalues, stable)
A = np.array([[4.0, 2.0],
              [2.0, 3.0]])

vals, Q = np.linalg.eigh(A)
Lambda = np.diag(vals)

print(np.allclose(A, Q @ Lambda @ Q.T)) # True

# Matrix square root A^{1/2}
A_sqrt = Q @ np.diag(np.sqrt(vals)) @ Q.T
print(np.allclose(A_sqrt @ A_sqrt, A)) # True
```

Not every matrix is diagonalizable. Repeated eigenvalues with too few eigenvectors produce defective matrices. SVD avoids this limitation and works on any matrix regardless of shape or rank.

A 3D rotation matrix R rotates every vector by 30 degrees around the z -axis. Does it have real eigenvectors? What does the characteristic polynomial look like?

QR decomposition

Any matrix $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ factors as:

$$A = QR$$

$Q \in \mathbb{R}^{m \times n}$ has orthonormal columns; $R \in \mathbb{R}^{n \times n}$ is upper triangular.

The columns of Q form an orthonormal basis for the column space of A . R stores the coefficients that reconstruct A 's columns in that basis.

QR decomposition is equivalent to the Gram-Schmidt process applied to the columns of A : successively project out the components already explained by previous columns.

```
A = np.array([[1, 2],
              [3, 4],
              [5, 6]])

Q, R = np.linalg.qr(A)
print(Q.shape)           # (3, 2)
print(R.shape)           # (2, 2) upper
                          triangular

print(np.allclose(A, Q @ R)) # True
print(np.allclose(Q.T @ Q, np.eye(2))) # True
```

QR solves least-squares problems more stably than the normal equations $(A^T A)x = A^T b$ because it avoids squaring the condition number.

QR iteration is also the standard algorithm for computing eigenvalues of large matrices.

Singular value decomposition

Every matrix $A \in \mathbb{R}^{m \times n}$ has a singular value decomposition:

$$A = U\Sigma V^T$$

- $U \in \mathbb{R}^{m \times m}$: orthogonal, columns are left singular vectors.
- $\Sigma \in \mathbb{R}^{m \times n}$: diagonal entries $\sigma_1 \geq \dots \geq \sigma_r \geq 0$ are singular values.
- $V \in \mathbb{R}^{n \times n}$: orthogonal, columns are right singular vectors.

The rank of A equals the number of strictly positive singular values.

SVD always exists. It works on rectangular, rank-deficient, and even the zero matrix.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

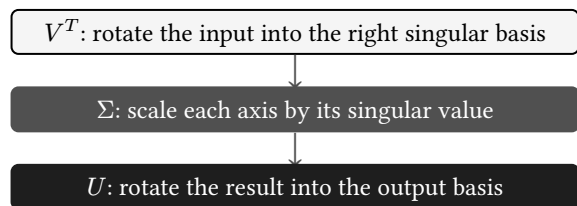
U, s, Vt = np.linalg.svd(A, full_matrices=True)
print(U.shape)    # (2, 2)
print(s)          # [9.508  0.773] -- singular values
print(Vt.shape)   # (3, 3)

# Reconstruct A
Sigma = np.zeros(A.shape)
Sigma[:2, :2] = np.diag(s)
print(np.allclose(A, U @ Sigma @ Vt)) # True
```

SVD is the most general matrix decomposition in numerical linear algebra. Eigendecomposition requires a square matrix; SVD does not.

SVD geometry

$A = U\Sigma V^T$ decomposes any linear map into three operations applied right to left.



Applied to the unit sphere: V^T reorients it, Σ stretches it into an ellipsoid, U reorients the ellipsoid.

The singular values σ_i measure how much A stretches each singular direction.

- Large σ_i : A amplifies signals in that direction.
- Small σ_i : A compresses signals in that direction.
- $\sigma_i = 0$: that direction is in the null space.

The condition number $\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$ now has a direct geometric reading: it is the ratio of the longest to the shortest axis of the output ellipsoid. A matrix with $\kappa \gg 1$ strongly amplifies some directions and crushes others, making the system sensitive to noise.

SVD applications: pseudoinverse and null space

PSEUDOINVERSE

The Moore-Penrose pseudoinverse A^+ generalizes the inverse to non-square and rank-deficient matrices.

$$A^+ = V\Sigma^+U^T$$

Σ^+ inverts each non-zero singular value and leaves zeros as zeros.

```
U, s, Vt = np.linalg.svd(A, full_matrices=False)
s_inv = np.where(s > 1e-10, 1.0 / s, 0.0)
A_plus = Vt.T @ np.diag(s_inv) @ U.T
# or: A_plus = np.linalg.pinv(A)
```

A^+b gives the minimum-norm least-squares solution to $Ax = b$.

NULL SPACE AND NUMERICAL RANK

The columns of V corresponding to $\sigma_i < \varepsilon$ span the null space of A .

The numerical rank is the count of singular values above the threshold. This is more stable than computing a determinant or counting non-zero pivots.

```
_, s, Vt = np.linalg.svd(A)
eps = 1e-10
rank = np.sum(s > eps)
null_V = Vt[s < eps].T # null space basis
```

The homogeneous system $Ax = \mathbf{0}$ appears in DLT camera calibration and the 8-point algorithm (will come in MVG lectures). The solution is the null space vector of A , found directly from SVD.

Least squares

When $A\mathbf{x} = \mathbf{b}$ has no exact solution, minimize the squared residual:

$$\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|_2^2$$

Setting the gradient to zero gives the normal equations:

$$A^T A \mathbf{x} = A^T \mathbf{b}$$

$$\mathbf{x}^* = (A^T A)^{-1} A^T \mathbf{b} = A^+ \mathbf{b}$$

When A has full column rank, $A^T A$ is invertible and the solution is unique.

WEIGHTED LEAST SQUARES

When measurements have different noise levels, weight each equation by its inverse variance:

$$\min_{\mathbf{x}} (A\mathbf{x} - \mathbf{b})^T W (A\mathbf{x} - \mathbf{b})$$

$$\mathbf{x}^* = (A^T W A)^{-1} A^T W \mathbf{b}$$

The weight matrix $W = \Sigma^{-1}$ is the inverse measurement covariance. This is the MAP estimator from probability lecture in matrix form.

Every sensor fusion algorithm reduces to a weighted least squares problem. The covariance matrices from probability lecture supply the weights.

Low-rank approximation

The rank- k truncated SVD:

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

The Eckart-Young theorem: A_k is the closest rank- k matrix to A in both the Frobenius norm and the spectral norm.

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2$$

Discarding the smallest singular values removes noise while retaining the dominant structure.

```
U, s, Vt = np.linalg.svd(A, full_matrices=False)

k = 2
A_k = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]

error = np.linalg.norm(A - A_k, 'fro')
# equals sqrt(sum of squared discarded singular values)

# Fraction of variance explained by rank k
explained = s[:k]**2 / np.sum(s**2)
print(explained.sum())
```

Applications: image compression, noise reduction in sensor data, approximate solutions to large SLAM systems, dimensionality reduction before learning.

**A matrix has singular values $[100, 50, 0.001, 0.0001]$.
What is its numerical rank? What fraction of the
variance does the rank-2 approximation keep?**

Jacobians

A non-linear function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be approximated linearly near a point x_0 :

$$f(x_0 + \delta x) \approx f(x_0) + J_f \delta x$$

The Jacobian $J_f \in \mathbb{R}^{m \times n}$ collects all first partial derivatives:

$$(J_f)_{ij} = \frac{\partial f_i}{\partial x_j}$$

Column j of J is the direction the output moves when input j increases by an infinitesimal amount.

```
def f(x):
    return np.array([x[0]**2 + x[1], x[0] * x[1]])

x0 = np.array([1.0, 2.0])
eps = 1e-6
J = np.zeros((2, 2))

for j in range(2):
    e = np.zeros(2)
    e[j] = eps
    J[:, j] = (f(x0 + e) - f(x0)) / eps

print(J) # [[2, 1], [2, 1]]
# analytic: [[2x0, 1], [x1, x0]] at (1,2)
```

The Jacobian is the matrix version of the derivative. Gauss-Newton and Levenberg-Marquardt (L5) approximate the Hessian with $J^T J$ and solve a sequence of linear systems of that form.

Covariance propagation

The covariance matrix of a random vector $\mathbf{x} \in \mathbb{R}^n$ is:

$$\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T]$$

It is always symmetric and positive semi-definite. Its SVD gives the principal axes (eigenvectors) and variances (eigenvalues) of the uncertainty ellipsoid from probability lecture.

The information matrix Σ^{-1} is also symmetric positive definite. It is the weight matrix in the weighted least squares from slide 25, which is the MAP estimator from probability lecture.

PROPAGATING UNCERTAINTY THROUGH A LINEAR MAP

If $\mathbf{y} = A\mathbf{x}$ and $\mathbf{x}$ has covariance Σ_x :

$$\Sigma_y = A\Sigma_x A^T$$

This is the Kalman filter predict step: the motion model A inflates the covariance.

For a non-linear map $\mathbf{y} = \mathbf{f}(\mathbf{x})$, linearize with the Jacobian:

$$\Sigma_y \approx J\Sigma_x J^T$$

This is the extended Kalman filter covariance propagation.

probability lecture introduced Σ as a belief; this lecture shows it is a symmetric positive definite matrix whose eigenstructure determines the shape of that belief.

Reference materials

TEXTBOOKS

Gilbert Strang: Introduction to Linear Algebra. Chapters 1 through 7 cover vectors, matrices, eigenvalues, and SVD with geometric motivation throughout. [Course page](#).

Timothy Sauer: Numerical Analysis. Covers condition numbers, QR, and SVD from a computation standpoint with error analysis.

LECTURES AND INTERACTIVE RESOURCES

Gilbert Strang: MIT 18.06 Linear Algebra. Full lecture series, freely available. [MIT OpenCourseWare](#).

3Blue1Brown: Essence of Linear Algebra. Geometric visual intuition for every topic in this session. [YouTube playlist](#).

Roger Labbe: Kalman and Bayesian Filters in Python. Shows covariance propagation and the Jacobian in working filter code. [GitHub](#).